

Московский Авиационный Институт
(Национальный Исследовательский Университет)
Институт №8 “Компьютерные науки и прикладная математика”
Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №1 по курсу
«Операционные системы»

Группа: М8О-215Б-23

Студент: Аралов И.А.

Преподаватель: Миронов Е.С.

Оценка: _____

Дата: 03.03.24

Москва, 2024

Постановка задачи

Вариант 1.

Родительский процесс создает дочерний процесс. Первой строчкой пользователь в консоль родительского процесса пишет имя файла, которое будет передано при создании дочернего процесса. Родительский и дочерний процесс должны быть представлены разными программами. Родительский процесс передает команды пользователя через `pipe1`, который связан с стандартным входным потоком дочернего процесса. Дочерний процесс при необходимости передает данные в родительский процесс через `pipe2`. Результаты своей работы дочерний процесс пишет в созданный им файл. Допускается просто открыть файл и писать туда, не перенаправляя стандартный поток вывода. Пользователь вводит команды вида: «число число число». Далее эти числа передаются от родительского процесса в дочерний. Дочерний процесс считает их сумму и выводит её в файл. Числа имеют тип `int`. Количество чисел может быть произвольным.

Общий метод и алгоритм решения

Использованные системные вызовы:

- `fork()` – создает дочерний процесс.
- `pipe()` – создает канал для обмена данными между процессами
- `execl()` – заменяет текущий процесс новым
- `close()` – закрывает файловый дескриптор
- `dup2()` – перенаправляет дескриптор
- `wait()` – ожидает завершение дочернего процесса

Алгоритм работы программы

Программа состоит из двух частей: родительского процесса (`parent.c`) и дочернего процесса (`child.c`). Они взаимодействуют через межпроцессное взаимодействие (IPC) с использованием `pipe`. Ниже приведен пошаговый алгоритм работы программы.

1. Родительский процесс (`parent.c`):

Создание `pipe`:

Создаются два канала (`pipe`): `fd1` и `fd2`.

`fd1` используется для передачи данных от родительского процесса к дочернему.

`fd2` используется для передачи данных от дочернего процесса к родительскому (хотя в данной программе он не используется).

Создание дочернего процесса:

Вызывается `fork()`, чтобы создать дочерний процесс.

Если `fork()` завершается с ошибкой, программа завершается.

Родительский процесс:

Закрываются ненужные концы pipe:

fd1[0] (чтение из первого pipe) закрывается, так как родительский процесс будет только писать в этот pipe.

fd2[1] (запись во второй pipe) закрывается, так как родительский процесс не будет использовать этот pipe для записи.

Пользователю предлагается ввести числа в формате: «число число число<newline>».

Введенная строка считывается с помощью fgets().

Строка разбивается на токены с помощью strtok(), и каждое число преобразуется в целое с помощью atoi().

Массив чисел сохраняется в numbers, где numbers[0] содержит количество введенных чисел, а остальные элементы — сами числа.

Массив чисел передается в дочерний процесс через fd1[1] с помощью write().

Ожидание завершения дочернего процесса:

Родительский процесс ожидает завершения дочернего процесса с помощью waitpid().

Если дочерний процесс завершился с ошибкой, родительский процесс выводит сообщение об ошибке.

Завершение работы:

Закрываются оставшиеся концы pipe: fd1[1] и fd2[0].

Программа завершает работу.

2. Дочерний процесс (child.c):

Перенаправление стандартного ввода и вывода:

Закрываются ненужные концы pipe:

fd1[1] (запись в первый pipe) закрывается, так как дочерний процесс будет только читать из этого pipe.

fd2[0] (чтение из второго pipe) закрывается, так как дочерний процесс не будет использовать этот pipe для чтения.

Стандартный ввод (STDIN_FILENO) перенаправляется на fd1[0] с помощью dup2().

Стандартный вывод (STDOUT_FILENO) перенаправляется на fd2[1] с помощью dup2() (хотя в данной программе это не используется).

Запуск дочернего процесса:

Вызывается exec1(), чтобы заменить текущий процесс на выполнение программы child.

Чтение данных:

Дочерний процесс читает массив чисел из стандартного ввода (который был перенаправлен на fd1[0]).

Данные сохраняются в массив numbers.

Вычисление суммы:

Вызывается функция sum_array(), которая вычисляет сумму чисел в массиве numbers.

Результат сохраняется в переменную sum.

Запись результата в файл:

Открывается файл result.txt для записи.

Сумма форматируется в строку с помощью sprintf().

Строка записывается в файл result.txt.

Файл закрывается.

Завершение работы:

Дочерний процесс завершает работу.

3. Взаимодействие между процессами:

Родительский процесс передает массив чисел дочернему процессу через pipe.

Дочерний процесс читает данные из pipe, вычисляет сумму и записывает результат в файл result.txt.

Код программы

parent.c

```
#include <stdio.h>

#include <stdlib.h>
```

```
#include <string.h>

#include <unistd.h>

#include <sys/types.h>

#include <sys/wait.h>

#include <fcntl.h>


#define MAX_SIZE 100

int main(){

    int fd1[2], fd2[2];


    if (pipe(fd1) == -1 || pipe(fd2) == -1) {

        printf("Ошибка при создании pipe\n");

        return 1;

    }


    pid_t pid = fork();

    if (pid == -1) {

        printf("Ошибка при создании дочернего процесса\n");

        return 1;

    }


    if (pid > 0) { // Родительский процесс

        close(fd1[0]); // Закрываем чтение из первого pipe

        close(fd2[1]); // Закрываем запись во второй pipe


        int number, numbers[MAX_SIZE];

        int size = 0;

        char buffer[1024]; // Буфер для чтения строки
```

```
printf("Введите данные в формате: «число число число<newline>».\\n");

if (fgets(buffer, sizeof(buffer), stdin) == NULL) {
    fprintf(stderr, "Ошибка чтения ввода\\n");
    return 1;
}

char *token = strtok(buffer, " "); // Разделение строки на токены
while (token != NULL && size < MAX_SIZE) {
    numbers[++size] = atoi(token); // Преобразование токена в число и
сохранение в массив
    token = strtok(NULL, " ");
}
numbers[0] = size;

if (write(fd1[1], &numbers, sizeof(numbers)) == -1) {
    printf("Ошибка при передаче числа\\n");
    return 1;
}

// Ожидание завершения дочернего процесса
int status;
waitpid(pid, &status, 0);

if (WIFEXITED(status) && WEXITSTATUS(status) == 1) {
    fprintf(stderr, "Дочерний процесс завершил работу с ошибкой.\\n");
    return 1;
}

close(fd1[1]); // Закрываем запись в первый pipe
```

```
    close(fd2[0]); // Закрываем чтение из второго pipe
} else { // Дочерний процесс

    close(fd1[1]); // Закрываем запись в первый pipe

    close(fd2[0]); // Закрываем чтение из второго pipe


    // Перенаправление stdin и stdout на pipe

    if (dup2(fd1[0], STDIN_FILENO) == -1) {

        printf("Ошибка при перенаправлении stdin\n");

        return 1;

    }

    if (dup2(fd2[1], STDOUT_FILENO) == -1) {

        printf("Ошибка при перенаправлении stdout\n");

        return 1;

    }


    // Запуск дочернего процесса

    execl("./child", "child", NULL);

    printf("Ошибка при вызове execl\n");

    return 1;

}

printf("Программа завершила работу без ошибок\n");

return 0;
}
```

child.c

```
#include <stdio.h>

#include <unistd.h>

#include <fcntl.h>

#include <math.h>

#include <string.h>

#define MAX_SIZE 100

int sum_array(int *array, int size) {

    int sum = 0;

    for (int i = 0; i < size; i++) {

        sum += array[i + 1];

    }

    return sum;

}

int main() {

    int sum, numbers[MAX_SIZE];

    // Чтение числа из стандартного ввода

    if (read(STDIN_FILENO, &numbers, sizeof(numbers)) == -1) {

        fprintf(stderr, "Ошибка при чтении чисел\n");

        return 1;

    }

    sum = sum_array(numbers, numbers[0]);

    int file = open("result.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (file == -1) {
```



```
        fprintf(stderr, "Ошибка при открытии файла\n");

        return 1;
    }

    // Форматирование результата в строку

    char buffer[16];

    int n = snprintf(buffer, sizeof(buffer), "%d\n", sum);

    if (n <= 0) {

        perror("Ошибка при форматировании строки");

        close(file);

        return 1;
    }

    // Запись строки в файл

    if (write(file, buffer, n) == -1) {

        fprintf(stderr, "Ошибка при записи в файл\n");

        close(file);

        return 1;
    }

    close(file);

    return 0;
}
```

Протокол работы программы

Тестирование:

```
root@fa7703109f3c:/workspaces/OS_labs/lab1/src# ./parent
Введите данные в формате: «число число число<newline>».
1 2 3 4 5
Программа завершила работу без ошибок
root@fa7703109f3c:/workspaces/OS_labs/lab1/src# cat result.txt
15
```

Strace:

```
root@fa7703109f3c:/workspaces/OS_labs/lab1/src# strace -f ./parent
execve("./parent", [ "./parent" ], 0x7ffe2ce2eb88 /* 30 vars */) = 0
brk(NULL)                               = 0x1d36000
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) =
0x7f8a0de99000
access("/etc/ld.so.preload", R_OK)      = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
newfstatat(3, "", {st_mode=S_IFREG|0644, st_size=25258, ...}, AT_EMPTY_PATH) = 0
mmap(NULL, 25258, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7f8a0de92000
close(3)                                = 0
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\20t\2\0\0\0\0\0"... , 832) = 832
= 784 pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"... , 784, 64)
newfstatat(3, "", {st_mode=S_IFREG|0755, st_size=1922136, ...}, AT_EMPTY_PATH) = 0
= 784 pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"... , 784, 64)
mmap(NULL, 1970000, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7f8a0dcb1000
mmap(0x7f8a0dcd7000, 1396736, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE,
3, 0x26000) = 0x7f8a0dcd7000
mmap(0x7f8a0de2c000, 339968, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3,
0x17b000) = 0x7f8a0de2c000
mmap(0x7f8a0de7f000, 24576, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE,
3, 0x1ce000) = 0x7f8a0de7f000
mmap(0x7f8a0de85000, 53072, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS,
-1, 0) = 0x7f8a0de85000
close(3)                                = 0
mmap(NULL, 12288, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) =
0x7f8a0dcae000
arch_prctl(ARCH_SET_FS, 0x7f8a0dcae740) = 0
set_tid_address(0x7f8a0dcae10)          = 7662
set_robust_list(0x7f8a0dcae20, 24)      = 0
rseq(0x7f8a0dc000, 0x20, 0, 0x53053053) = 0
mprotect(0x7f8a0de7f000, 16384, PROT_READ) = 0
mprotect(0x403000, 4096, PROT_READ)     = 0
mprotect(0x7f8a0dec000, 8192, PROT_READ) = 0
prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024, rlim_max=RLIM64_INFINITY}) = 0
munmap(0x7f8a0de92000, 25258)           = 0
pipe2([3, 4], 0)                       = 0
```

```

pipe2([5, 6], 0) = 0
clone(child_stack=NULL, flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD, strace:
Process 7663 attached
<unfinished ...>
[pid 7663] set_robust_list(0x7f8a0dcaea20, 24 <unfinished ...>
[pid 7662] <... clone resumed>, child_tidptr=0x7f8a0dcaea10) = 7663
[pid 7663] <... set_robust_list resumed>) = 0
[pid 7662] close(3) = 0
[pid 7663] close(4 <unfinished ...>
[pid 7662] close(6 <unfinished ...>
[pid 7663] <... close resumed>) = 0
[pid 7662] <... close resumed>) = 0
[pid 7662] newfstatat(1, "", <unfinished ...>
[pid 7663] close(5 <unfinished ...>
[pid 7662] <... newfstatat resumed>{st_mode=S_IFCHR|0620, st_rdev=makedev(0x88, 0),
...}, AT_EMPTY_PATH) = 0
[pid 7663] <... close resumed>) = 0
[pid 7662] getrandom( <unfinished ...>
[pid 7663] dup2(3, 0 <unfinished ...>
[pid 7662] <... getrandom resumed>"\x97\xd4\x01\x83\x78\x26\x8d\x63", 8,
GRND_NONBLOCK) = 8
[pid 7663] <... dup2 resumed>) = 0
[pid 7662] brk(NULL <unfinished ...>
[pid 7663] dup2(6, 1 <unfinished ...>
[pid 7662] <... brk resumed>) = 0x1d36000
[pid 7663] <... dup2 resumed>) = 1
[pid 7662] brk(0x1d57000) = 0x1d57000
[pid 7663] execve("./child", ["child"], 0x7ffec332f4a8 /* 30 vars */ <unfinished ...>
[pid 7662] write(1, "\320\222\320\262\320\265\320\264\320\270\321\202\320\265

\320\264\320\260\320\275\320\275\321\213\320\265 \320\262 \321"... , 94Введите данные в
формате: «число число число<endline>».
) = 94
[pid 7662] newfstatat(0, "", {st_mode=S_IFCHR|0620, st_rdev=makedev(0x88, 0), ...},
AT_EMPTY_PATH) = 0
[pid 7662] read(0, <unfinished ...>
[pid 7663] <... execve resumed>) = 0
[pid 7663] brk(NULL) = 0x1e6e000
[pid 7663] mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) =
0x7fe4e3c66000
[pid 7663] access("/etc/ld.so.preload", R_OK) = -1 ENOENT (No such file or directory)
[pid 7663] openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 4
[pid 7663] newfstatat(4, "", {st_mode=S_IFREG|0644, st_size=25258, ...},
AT_EMPTY_PATH) = 0
[pid 7663] mmap(NULL, 25258, PROT_READ, MAP_PRIVATE, 4, 0) = 0x7fe4e3c5f000
[pid 7663] close(4) = 0
[pid 7663] openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 4
[pid 7663] read(4,
"\177ELF\2\1\1\3\0\0\0\0\0\0\3\0>\0\1\0\0\0\20t\2\0\0\0\0\0"... , 832) = 832

```

```

[pid 7663] pread64(4,
"\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"..., 784, 64) = 784
[pid 7663] newfstatat(4, "", {st_mode=S_IFREG|0755, st_size=1922136, ...},
AT_EMPTY_PATH) = 0
[pid 7663] pread64(4,
"\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"..., 784, 64) = 784
[pid 7663] mmap(NULL, 1970000, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 4, 0) =
0x7fe4e3a7e000
[pid 7663] mmap(0x7fe4e3aa4000, 1396736, PROT_READ|PROT_EXEC,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 4, 0x26000) = 0x7fe4e3aa4000
[pid 7663] mmap(0x7fe4e3bf9000, 339968, PROT_READ,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 4, 0x17b000) = 0x7fe4e3bf9000
[pid 7663] mmap(0x7fe4e3c4c000, 24576, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 4, 0x1ce000) = 0x7fe4e3c4c000
[pid 7663] mmap(0x7fe4e3c52000, 53072, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0x7fe4e3c52000
[pid 7663] close(4) = 0
[pid 7663] mmap(NULL, 12288, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) =
0x7fe4e3a7b000
[pid 7663] arch_prctl(ARCH_SET_FS, 0x7fe4e3a7b740) = 0
[pid 7663] set_tid_address(0x7fe4e3a7ba10) = 7663
[pid 7663] set_robust_list(0x7fe4e3a7ba20, 24) = 0
[pid 7663] rseq(0x7fe4e3a7c060, 0x20, 0, 0x53053053) = 0
[pid 7663] mprotect(0x7fe4e3c4c000, 16384, PROT_READ) = 0
[pid 7663] mprotect(0x403000, 4096, PROT_READ) = 0
[pid 7663] mprotect(0x7fe4e3c98000, 8192, PROT_READ) = 0
[pid 7663] prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024,
rlim_max=RLIM64_INFINITY}) = 0
[pid 7663] munmap(0x7fe4e3c5f000, 25258) = 0
[pid 7663] read(0, 1 2 3 4
<unfinished ...>
[pid 7662] <... read resumed>"1 2 3 4\n", 1024) = 8
[pid 7662] write(4,
"\4\0\0\0\1\0\0\0\2\0\0\0\3\0\0\0\4\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0"..., 400) = 400
[pid 7663] <... read
400 resumed>"\4\0\0\0\1\0\0\0\2\0\0\0\3\0\0\0\4\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0"..., 400) =
[pid 7662] wait4(7663, <unfinished ...>
[pid 7663] openat(AT_FDCWD, "result.txt", O_WRONLY|O_CREAT|O_TRUNC, 0644) = 4
[pid 7663] write(4, "10\n", 3) = 3
[pid 7663] close(4) = 0
[pid 7663] exit_group(0) = ?
[pid 7663] +++ exited with 0 +++
<... wait4 resumed>[{WIFEXITED(s) && WEXITSTATUS(s) == 0}], 0, NULL) = 7663
--- SIGCHLD {si_signo=SIGCHLD, si_code=CLD_EXITED, si_pid=7663, si_uid=0, si_status=0,
si_utime=0, si_stime=0} ---
close(4) = 0
close(5) = 0
write(1, "\320\237\321\200\320\276\320\263\321\200\320\260\320\274\320\274\320\260

```

```
\320\267\320\260\320\262\320\265\321\200\321\210\320"... , 71Программа завершила работу  
без ошибок  
) = 71  
exit_group(0) = ?  
+++ exited with 0 +++
```

Вывод

В ходе выполнения лабораторной работы была разработана программа, состоящая из двух процессов: родительского и дочернего. Программа демонстрирует использование механизмов межпроцессного взаимодействия (IPC) с помощью pipe, а также работу с файлами и перенаправлением стандартного ввода/вывода.